Presentation on

Introduction to OOP

Object and Class in C# ⇲

Presented By: Bikal Shrestha
Roll No: 173
Level: Masters of Information Communication Technology(M.Ed. ICT)
Course: ICT Ed. 535: Visual Programming
Central Department OF Education, Tribhuvan University

Presented to: Piyush Pant
Course: Subject Instructor

11th March, 2026

Another trick in software is to avoid rewriting the software by using a piece that's already been written, so called component approach which the latest for this in the most advanced form is what's called Object Oriented Programming

--Bill Gates--

Programming Paradigm

A programming paradigm is a style or approach used to design and organize programs.

Common paradigms:

- Procedural Programming{Step wise step
- Object-Oriented Programming{Classes and Objects}
- Functional Programming{Functions and mathematical expressions}
- Logic Programming{Rules and facts}

C# mainly supports Object-Oriented Programming (OOP).

Basic Concept

Object–Oriented Programming

Object-Oriented Programming (OOP) is a programming methodology that organizes software design around objects rather than functions and logic.

Objects contain:

- Data (Attributes / Fields)
- Behavior (Methods / Functions)

Key Idea

Combine data and functionality into a single unit called an object.

OOP

Why OOP is Important? ↘

- Code Reusability
- Modularity
- Maintainability
- Security
- Real-world modeling
- Scalability

Basic Concepts of OOP ↘

Core principles of OOP:

1. Class
2. Object
3. Encapsulation
4. Inheritance
5. Polymorphism
6. Abstraction

This presentation focuses mainly on:

- Classes
- Objects

What is a Class?

A class is a blueprint or template used to create objects.

It defines:

- Variables (Fields)
- Methods (Functions)
- Properties

Example Concept

Class → Car

Objects → Toyota, Honda, Tesla

OOP

Structure of a Class in C#

Basic syntax:

```
class ClassName
{
    // Fields
    // Properties
    // Methods
}
```

Example:

```
class Student
{
    public string name;
    public int age;
}
```

OOP

What is an Object?

An object is an instance of a class.

It represents a real-world entity.

Example:

- Student
- Car
- Book
- Employee

OOP

Object Creation Syntax

```
ClassName objectName = new ClassName();
```

Example:

```
Student s1 = new Student();
```

Example of Class and Object in C#

OOP

```
using System;

class Student
{
    public string name;
    public int age;
}

class Program
{
    static void Main()
    {
        Student s1 = new Student();

        s1.name = "Ram";
        s1.age = 22;

        Console.WriteLine("Name: " + s1.name);
        Console.WriteLine("Age: " + s1.age);
    }
}
```

Class with Methods

A class can contain methods that define behavior.

Example:

```
using System;

class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }
}

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();
        int result = c.Add(5, 3);

        Console.WriteLine("Result: " + result);
    }
}
```

OOP

Fields and Methods Together

Example:

OOP

```
using System;

class Person
{
   public string name;

   public void Greet()
   {
      Console.WriteLine("Hello, my name is " + name);
   }
}

class Program
{
   static void Main()
   {
      Person p1 = new Person();
      p1.name = "Bikal";

      p1.Greet();
   }
}
```

Real World Example (Bank Account)

Example:

OOP

```
using System;

class BankAccount
{
    public string accountHolder;
    public double balance;

    public void Deposit(double amount)
    {
        balance += amount;
    }

    public void Display()
    {
        Console.WriteLine("Account Holder: " +
accountHolder);
        Console.WriteLine("Balance: " + balance);
    }
}
```

Real World Example (Bank Account)

Example:

```
class Program
{
 static void Main()
 {
 BankAccount acc = new BankAccount();

 acc.accountHolder = "Ram";
 acc.balance = 5000;

 acc.Deposit(2000);

 acc.Display();
 }
}
```

OOP

Class vs Object

Class	Object
Blueprint	Instance of class
Logical entity	Physical entity
No memory allocation	Memory allocated
Example: Car	Example: Tesla

OOP

Advantages of Using Classes and Objects

- Code organization
- Reusability
- Easier debugging
- Better program structure
- Real-world modeling

Example:

Large systems like:

- Banking software
- Hospital systems
- E-commerce systems

References

- Microsoft Visual Studio Documentation – Microsoft Learn
- https://learn.microsoft.com
- C# Programming Guide – Microsoft Documentation.

Thank you! Open to feedback!

reach out for any questions ↘

email

bik.mict@gmail.com

website

www.bikalsth.com.np